

CLAUDE CODE MASTER BUILD PROMPT

Medical School Scholarship Management System

You are acting as the lead software architect, senior full-stack engineer, product manager, UX designer, security engineer, database architect, QA engineer, and technical writer for this project.

Your task is to design and build a production-quality web application called:

Medical School Scholarship Management System

The application will help premedical and medical students discover, evaluate, organize, apply for, and win scholarships while reducing educational debt and avoiding missed deadlines.

This application is specifically focused on scholarship management. Do not expand it into a general physician operating system, residency application platform, or medical-practice-management system unless instructed in a future phase.

The system should be useful for one student initially but should be architected so it could later support many users.

1. CORE PRODUCT VISION

The application should function like a scholarship CRM, application project manager, document library, essay workspace, deadline calendar, and financial-impact dashboard.

The application should help the user answer:

1. Which scholarships am I eligible for?
2. Which scholarships should I apply for first?
3. What is due next?
4. Which application materials are missing?
5. Which essays can be reused or adapted?
6. How much scholarship funding have I applied for?
7. How much have I won?
8. How much tuition and student-loan interest might I avoid?

9. Which scholarship categories produce the best return on effort?

10. What should I work on today?

The product should reduce the organizational burden of applying for many scholarships over several years.

2. PRIMARY USER

The initial user is:

- A premedical or medical student.
- Attending or planning to attend medical school in the United States.
- Interested in applying for institutional, local, state, national, private, professional-association, hospital-foundation, community-foundation, and service-oriented scholarships.
- Willing to apply consistently over multiple years.
- Interested in minimizing debt while preserving future career flexibility.
- Likely to pursue Family Medicine but should not be restricted to that specialty.
- May eventually pursue Direct Primary Care, but that is outside the scope of this initial application.

The product should remain broadly useful for students pursuing any medical specialty.

3. PRODUCT PRINCIPLES

Follow these principles throughout development:

1. Build incrementally.
2. Keep each phase usable.
3. Do not build later-phase features prematurely.
4. Do not sacrifice reliability for feature count.
5. Use clear and maintainable architecture.
6. Protect sensitive personal and financial information.

7. Keep the user in control of all AI-generated content.
 8. Never submit scholarship applications automatically.
 9. Never send emails automatically without explicit confirmation.
 10. Clearly distinguish verified scholarship information from AI-generated suggestions.
 11. Make deadlines and eligibility rules easy to audit.
 12. Preserve the user's authentic voice in essays.
 13. Make all financial projections visibly dependent on assumptions.
 14. Avoid fabricated scholarship details.
 15. Design for accessibility and mobile responsiveness.
-

4. RECOMMENDED TECHNOLOGY STACK

Use the following stack unless a compelling technical reason requires a change.

Frontend

- Next.js using the App Router
- TypeScript
- React
- Tailwind CSS
- shadcn/ui
- React Hook Form
- Zod
- TanStack Table
- Recharts
- Lucide icons

Backend

- Next.js server actions and route handlers where appropriate
- Supabase PostgreSQL

- Supabase Authentication
- Supabase Storage
- Row Level Security
- PostgreSQL full-text search
- pgvector for later semantic search features

AI

- Create a provider abstraction.
- Default provider: Anthropic Claude API
- Optional provider: OpenAI API
- AI features must be disabled gracefully when no API key is configured.
- Never expose API keys to the browser.

Testing

- Vitest
- React Testing Library
- Playwright
- ESLint
- TypeScript strict mode

Deployment

- Vercel
- Supabase
- GitHub Actions

Optional Later Integrations

Do not implement these until their assigned phase:

- Google Calendar
- Microsoft Outlook Calendar
- Gmail

- Microsoft Outlook Email
 - Twilio
 - Scholarship discovery via search APIs
 - OCR or document extraction services
-

5. DEVELOPMENT PROCESS

Do not attempt to build the entire application at once.

For every phase:

1. Review the phase requirements.
2. Create or update the implementation plan.
3. Identify database changes.
4. Identify new routes and components.
5. Implement the smallest complete version.
6. Add validation and error handling.
7. Add automated tests.
8. Run linting, type checking, and tests.
9. Fix all critical errors.
10. Update documentation.
11. Create a concise completion report.
12. Stop and wait for approval before beginning the next phase.

At the end of every phase, provide:

- What was completed.
- Files created or modified.
- Database migrations added.
- Tests added.
- Commands that were run.

- Known limitations.
- Recommended next step.
- Any manual configuration required.

Do not silently skip requirements. If a feature cannot be completed, explain why and implement the safest functional alternative.

6. CODE QUALITY REQUIREMENTS

Use the following standards:

- TypeScript strict mode.
- No unnecessary use of any.
- Reusable typed components.
- Clear domain models.
- Server-side authorization checks.
- Zod validation for all external or user-supplied data.
- Clear loading, empty, success, and error states.
- Accessible labels and keyboard navigation.
- Meaningful error messages.
- Avoid deeply nested components.
- Avoid oversized files.
- Separate business logic from presentation.
- Centralize financial calculations.
- Centralize scholarship-ranking calculations.
- Document non-obvious logic.
- Use database transactions where partial updates could cause inconsistency.
- Use migrations for every schema change.
- Never rely solely on client-side security.

- Never log secrets, essay content, financial data, or private documents unnecessarily.
-

7. APPLICATION INFORMATION ARCHITECTURE

The eventual application should include these primary navigation areas:

1. Dashboard
2. Scholarships
3. Applications
4. Tasks
5. Calendar
6. Essays
7. Documents
8. Recommenders
9. Experiences
10. Financial Impact
11. Reports
12. Profile
13. Settings

Not all sections will be implemented in the first phase.

8. CORE DATA ENTITIES

Design the database around the following entities.

User Profile

Fields should eventually include:

- ID
- User ID

- Full name
- Preferred name
- Email
- Phone
- Current state of residence
- Permanent state of residence
- County
- Citizenship status
- Medical-school status
- Medical school
- Expected matriculation year
- Expected graduation year
- Current medical-school year
- Undergraduate school
- Undergraduate graduation year
- Intended specialty
- Rural background indicator
- First-generation student indicator
- Financial-need indicator
- Military affiliation
- Service interests
- Leadership interests
- Research interests
- Geographic preferences
- Biography
- Career goals

- Created at
- Updated at

Sensitive demographic fields should be optional. Do not infer them.

Scholarship

Fields should eventually include:

- ID
- Name
- Sponsoring organization
- Description
- Official URL
- Application URL
- Contact name
- Contact email
- Contact phone
- Scholarship category
- Award type
- Minimum award
- Maximum award
- Typical award
- Number of awards
- Renewable
- Renewal period
- Renewal requirements
- Deadline
- Deadline type
- Opens date

- Academic-year applicability
- Eligible medical-school years
- Citizenship restrictions
- State restrictions
- County restrictions
- School restrictions
- Specialty restrictions
- Demographic eligibility notes
- Financial-need requirement
- Merit requirement
- Service requirement
- Service-obligation details
- Membership requirement
- Essay required
- Essay prompts
- Recommendation letters required
- Transcript required
- FAFSA required
- Financial documents required
- Interview required
- Estimated hours to complete
- Competitiveness level
- Source type
- Verification status
- Last verified date
- Last verified by

- Recurrence pattern
- Tags
- Notes
- Created by
- Created at
- Updated at
- Archived

Scholarship Application

Fields should eventually include:

- ID
- User ID
- Scholarship ID
- Application cycle
- Status
- Priority
- Intent to apply
- Date started
- Date submitted
- Submission confirmation
- Decision date
- Decision status
- Amount awarded
- Amount accepted
- Award payment date
- Renewal status
- Eligibility confidence

- User fit score
- Estimated probability
- Expected value
- Estimated hours
- Actual hours
- Notes
- Created at
- Updated at

Application Requirement

- ID
- Scholarship application ID
- Requirement type
- Requirement title
- Description
- Required
- Completed
- Due date
- Linked essay ID
- Linked document ID
- Linked recommender ID
- Completion date
- Notes

Task

- ID
- User ID
- Scholarship application ID

- Title
- Description
- Status
- Priority
- Due date
- Completed date
- Estimated minutes
- Actual minutes
- Task category
- Created at
- Updated at

Essay

- ID
- User ID
- Title
- Essay category
- Current content
- Word count
- Character count
- Target word count
- Target character count
- Voice notes
- Reusable
- Version number
- Created at
- Updated at

Essay Version

- ID
- Essay ID
- Version number
- Content
- Change summary
- Created by
- Created at

Essay Prompt

- ID
- Scholarship application ID
- Prompt text
- Word limit
- Character limit
- Linked essay ID
- Draft status
- Finalized
- Notes

Document

- ID
- User ID
- Name
- Document type
- Storage path
- MIME type
- File size

- Description
- Issue date
- Expiration date
- Verified
- Tags
- Created at
- Updated at

Recommender

- ID
- User ID
- Full name
- Title
- Organization
- Email
- Phone
- Relationship
- Relationship start date
- Areas they can address
- Preferred contact method
- Notes
- Created at
- Updated at

Recommendation Request

- ID
- Recommender ID
- Scholarship application ID

- Date requested
- Due date
- Status
- Date received
- Reminder date
- Thank-you sent
- Notes

Experience

- ID
- User ID
- Experience type
- Organization
- Role
- Start date
- End date
- Ongoing
- Hours
- Description
- Impact
- Leadership component
- Clinical component
- Research component
- Community service component
- Tags
- Created at
- Updated at

Financial Assumption

- ID
- User ID
- Tuition per year
- Years remaining
- Existing scholarship amount
- Existing debt
- Expected borrowing rate
- Loan interest rate
- Capitalization assumptions
- Repayment term
- Expected annual tuition increase
- Notes
- Created at
- Updated at

Financial Projection

- ID
- User ID
- Projection date
- Total scholarship awards
- Estimated borrowing avoided
- Estimated interest avoided
- Projected graduation debt
- Baseline graduation debt
- Projection assumptions
- Created at

Activity Log

- ID
 - User ID
 - Entity type
 - Entity ID
 - Action
 - Description
 - Timestamp
-

9. STATUS ENUMERATIONS

Use consistent enumerations.

Scholarship Application Status

- Discovered
- Reviewing
- Eligible
- Possibly Eligible
- Ineligible
- Interested
- Planning
- Preparing
- Waiting on Materials
- Drafting Essays
- Ready for Review
- Ready to Submit
- Submitted
- Interview

- Awarded
- Not Awarded
- Withdrawn
- Renewal Pending
- Renewed
- Archived

Task Status

- Not Started
- In Progress
- Blocked
- Completed
- Canceled

Priority

- Critical
- High
- Medium
- Low

Verification Status

- Unverified
- User Verified
- AI Assisted Review
- Official Source Verified
- Outdated
- Needs Reverification

10. PHASE 0 — PRODUCT DEFINITION AND PROJECT SETUP

Objective

Create the technical and product foundation before feature development.

Required Work

1. Create a concise product requirements document.
2. Create a technical architecture document.
3. Create a database entity relationship outline.
4. Create a route map.
5. Create a component map.
6. Create a security model.
7. Create a development roadmap.
8. Initialize the Next.js project.
9. Configure TypeScript strict mode.
10. Configure Tailwind CSS.
11. Install shadcn/ui.
12. Configure ESLint and Prettier.
13. Configure Vitest.
14. Configure Playwright.
15. Create environment-variable documentation.
16. Create a .env.example.
17. Create Git ignore rules.
18. Create an initial README.
19. Create a basic application shell.
20. Add a responsive sidebar and top navigation.
21. Add light mode and dark mode.
22. Create placeholder pages for future modules.
23. Create a design-token system.

24. Establish consistent typography and spacing.

25. Add a basic error boundary and 404 page.

Routes

Create placeholder routes for:

- /
- /dashboard
- /scholarships
- /applications
- /tasks
- /calendar
- /essays
- /documents
- /recommenders
- /experiences
- /financial-impact
- /reports
- /profile
- /settings

Acceptance Criteria

- The project installs successfully.
- The development server runs.
- The sidebar works on desktop and mobile.
- All routes load.
- Light and dark modes work.
- Type checking passes.
- Linting passes.

- Basic tests pass.
- No Supabase functionality is required yet.
- Documentation clearly explains how to run the project.

Stop after Phase 0 and provide a completion report.

11. PHASE 1 — AUTHENTICATION AND USER PROFILE

Objective

Allow a user to create an account, sign in securely, and establish a scholarship profile.

Required Work

1. Configure Supabase.
2. Add authentication.
3. Support email and password sign-up.
4. Support email and password login.
5. Support password reset.
6. Support logout.
7. Add protected routes.
8. Create the profile table.
9. Enable Row Level Security.
10. Ensure users can access only their own profile.
11. Build a first-time onboarding flow.
12. Build a profile editor.
13. Add profile completion percentage.
14. Add validation.
15. Add autosave or clear save behavior.
16. Add success and error notifications.
17. Add profile fields that influence scholarship eligibility.

18. Allow users to skip optional sensitive questions.

19. Explain how profile data will be used.

20. Add an account-deletion workflow or placeholder with clear instructions.

Onboarding Sections

Personal

- Full name
- Preferred name
- Email
- State
- County
- Citizenship status

Education

- Premedical, accepted, or enrolled status
- Medical school
- Expected matriculation year
- Expected graduation year
- Current medical-school year

Career

- Intended specialty
- Interest in primary care
- Interest in underserved practice
- Geographic preferences
- Long-term career goals

Eligibility

- Rural background
- First-generation background

- Financial need
- Military connection
- Community-service focus
- Leadership experience
- Research experience

All optional sensitive fields must be clearly labeled optional.

Acceptance Criteria

- A new user can register.
- A registered user can sign in and sign out.
- Protected pages redirect unauthenticated users.
- A user can complete onboarding.
- A user can edit their profile.
- Users cannot read or modify another user's data.
- Row Level Security is tested.
- Form validation is tested.

Stop after Phase 1.

12. PHASE 2 — SCHOLARSHIP DATABASE

Objective

Create a reliable scholarship database with manual entry, editing, searching, filtering, and verification.

Required Work

1. Create scholarship database tables.
2. Add Row Level Security.
3. Decide how shared scholarships and private user-created scholarships should work.
4. Implement a practical initial model:

- System scholarships can be visible to all users.
 - User-created scholarships can be private.
- 5. Create a scholarship list page.
- 6. Create a scholarship detail page.
- 7. Create scholarship creation and editing forms.
- 8. Add archive functionality.
- 9. Add soft deletion where appropriate.
- 10. Add duplication functionality.
- 11. Add search.
- 12. Add filters.
- 13. Add sorting.
- 14. Add pagination.
- 15. Add saved views if reasonable.
- 16. Add verification status.
- 17. Add last-verified date.
- 18. Add source URL.
- 19. Add warnings for stale scholarship records.
- 20. Add CSV import.
- 21. Add CSV export.
- 22. Add sample data for local development.
- 23. Add duplicate-detection warnings.
- 24. Add scholarship tags.
- 25. Add notes.

Required Filters

- Deadline range
- Award range

- State
- County
- Medical-school year
- Specialty
- Citizenship
- Need based
- Merit based
- Service commitment
- Renewable
- Essay required
- Recommendation required
- Verification status
- Scholarship category
- Archived status

Scholarship Detail Page

Display:

- Scholarship name
- Organization
- Award range
- Deadline
- Eligibility summary
- Requirements
- Essay prompts
- Recommendation requirements
- Official links
- Verification information

- Notes
- Tags
- Application status
- Create-application button

Data Reliability Rules

- Do not present an unverified deadline as guaranteed.
- Display source and verification status.
- Clearly show last verified date.
- Warn users to confirm details on the official website.
- Do not fabricate missing scholarship details.

Acceptance Criteria

- Users can create, edit, view, archive, import, and export scholarships.
- Search and filters work.
- Validation prevents invalid award ranges and malformed URLs.
- Stale scholarships are visibly flagged.
- Duplicate warnings work.
- Tests cover core CRUD and filter logic.

Stop after Phase 2.

13. PHASE 3 — APPLICATION PIPELINE

Objective

Turn scholarship records into trackable scholarship applications.

Required Work

1. Create scholarship application tables.
2. Allow users to create an application from a scholarship.
3. Add application cycle.

4. Add status.
5. Add priority.
6. Add expected award value.
7. Add estimated time.
8. Add user notes.
9. Add application requirements.
10. Add checklist functionality.
11. Add kanban view.
12. Add table view.
13. Add list view.
14. Support drag-and-drop status changes.
15. Add application detail page.
16. Add status history.
17. Add application activity log.
18. Add submission confirmation.
19. Add decision tracking.
20. Add award tracking.
21. Add renewal tracking.
22. Prevent accidental duplicate applications for the same cycle.
23. Allow a duplicate when intentionally applying in a new cycle.

Application Detail Sections

- Overview
- Eligibility
- Requirements
- Essays
- Documents

- Recommendations
- Tasks
- Timeline
- Submission
- Decision
- Award
- Notes
- Activity

Acceptance Criteria

- A scholarship can be converted into an application.
- Applications can move through statuses.
- Requirements can be checked off.
- The pipeline updates in real time or after reliable refresh.
- Duplicate-cycle warnings work.
- The activity log records major changes.
- Award amounts update dashboard calculations later.

Stop after Phase 3.

14. PHASE 4 — TASKS AND DEADLINE MANAGEMENT

Objective

Create a dependable task and deadline system.

Required Work

1. Create task tables.
2. Allow tasks to be linked to applications.
3. Allow independent tasks.
4. Add due dates.

5. Add priorities.
6. Add statuses.
7. Add estimated time.
8. Add actual time.
9. Add recurring tasks if reasonable.
10. Add task list.
11. Add task board.
12. Add “Today” view.
13. Add “This Week” view.
14. Add “Overdue” view.
15. Add “Blocked” view.
16. Add quick-add.
17. Add bulk complete.
18. Add bulk reschedule.
19. Generate default tasks when an application is created.
20. Allow users to modify generated tasks.
21. Add deadline countdowns.
22. Add warnings based on unfinished requirements.
23. Add a calendar view.
24. Add notification-preference placeholders.
25. Add in-app reminders.

Default Application Tasks

When an application is created, optionally generate:

- Verify official deadline
- Confirm eligibility
- Review requirements

- Request transcript
- Request recommendation letters
- Select essay drafts
- Draft essays
- Revise essays
- Proofread application
- Confirm attachments
- Submit application
- Save submission confirmation
- Send recommender thank-you notes
- Record decision

Deadline Risk Levels

Create a deadline-risk indicator based on:

- Days remaining
- Number of incomplete requirements
- Estimated work remaining
- Missing recommendations
- Missing essays
- Missing documents

Use clear labels such as:

- On Track
- Attention Needed
- At Risk
- Critical

Do not present the risk score as certainty.

Acceptance Criteria

- Tasks can be created, edited, completed, and linked.
- Overdue and upcoming tasks display correctly.
- Default application tasks can be generated.
- Deadline-risk logic has unit tests.
- Calendar view displays deadlines and tasks.

Stop after Phase 4.

15. PHASE 5 — DASHBOARD AND PRIORITIZATION

Objective

Build a useful daily command center.

Dashboard Sections

Financial Snapshot

- Total scholarship amount available
- Total amount planned
- Total amount applied for
- Total amount pending
- Total amount awarded
- Total amount accepted
- Estimated tuition reduction
- Estimated debt avoided

Pipeline Snapshot

- Number of discovered opportunities
- Number being reviewed
- Number in preparation
- Number submitted
- Number awarded

- Number not awarded
- Win rate

Deadline Snapshot

- Due today
- Due this week
- Due this month
- Overdue tasks
- Applications at risk

Priority Queue

Show the highest-priority applications based on:

- Deadline proximity
- Eligibility confidence
- Award amount
- Estimated effort
- Application readiness
- Verification status
- Strategic value

Today's Work

Display:

- Tasks due today
- Overdue tasks
- Recommended next actions
- Applications needing attention

Progress

- Profile completion
- Essay-library readiness

- Document completeness
- Recommender readiness
- Applications submitted this month
- Scholarship dollars applied for this month

Priority Score

Create a transparent, configurable prioritization formula.

Example inputs:

- Eligibility confidence: 0–100
- Deadline urgency: 0–100
- Award value: normalized
- Estimated effort: inverse score
- Application readiness: 0–100
- Scholarship competitiveness: 0–100
- Verification confidence: 0–100
- Strategic alignment: 0–100

Create a documented default formula. Show users the factors behind the score.

Do not claim the score predicts actual award outcomes.

Acceptance Criteria

- Dashboard calculations use real application data.
- Empty states guide new users.
- Priority calculations are tested.
- The user can understand why an application is prioritized.
- Financial totals reconcile with application records.

Stop after Phase 5.

16. PHASE 6 — ESSAY LIBRARY

Objective

Create a reusable essay-management workspace.

Required Work

1. Create essay and essay-version tables.
2. Create essay categories.
3. Create an essay editor.
4. Add autosave with safe conflict handling.
5. Add manual version creation.
6. Add version history.
7. Add word count.
8. Add character count.
9. Add target limits.
10. Add reusable tags.
11. Link essays to scholarship prompts.
12. Allow one essay to support multiple prompts.
13. Allow prompt-specific copies.
14. Add compare versions.
15. Add restore version.
16. Add search.
17. Add filters.
18. Add favorite or core essay designation.
19. Add notes about tone and authentic voice.
20. Add export to plain text, Markdown, and DOCX if practical.

Essay Categories

Include:

- Why Medicine

- Career Goals
- Primary Care
- Family Medicine
- Community Service
- Leadership
- Adversity
- Resilience
- Research
- Innovation
- Ethics
- Diversity and Perspective
- Rural Health
- Underserved Communities
- Financial Need
- Academic Achievement
- Personal Growth
- Failure and Learning
- Future Impact
- General Personal Statement
- Custom

AI Features for This Phase

Create an AI-assistance panel that can:

- Summarize an essay.
- Identify the main theme.
- Suggest areas needing more specificity.
- Identify unsupported claims.

- Suggest where examples could strengthen the essay.
- Reduce word count.
- Expand underdeveloped sections.
- Adapt an existing essay to a prompt.
- Compare an essay with a prompt.
- Check whether the prompt has been answered.
- Suggest an outline.
- Proofread grammar and clarity.
- Preserve the user's voice.

AI Safety Rules

- Never overwrite the original essay without confirmation.
- Always create suggestions or a new version.
- Clearly label AI-generated content.
- Never invent personal experiences.
- Use placeholders when facts are missing.
- Do not fabricate achievements, service hours, awards, hardship, demographic identity, or patient interactions.
- Warn the user to verify every generated statement.
- Do not send or submit essays automatically.

Acceptance Criteria

- Essays can be created, saved, versioned, linked, compared, and restored.
- Word and character limits work.
- AI features fail gracefully when no API key exists.
- Original content is preserved.
- Tests cover versioning and limit calculations.

Stop after Phase 6.

17. PHASE 7 — DOCUMENT VAULT

Objective

Securely organize scholarship materials.

Document Types

- CV
- Resume
- Transcript
- Enrollment verification
- Acceptance letter
- Financial-aid document
- FAFSA-related document
- Tax document
- Recommendation letter
- Certification
- Award documentation
- Volunteer verification
- Research documentation
- Identification
- Photograph
- Essay attachment
- Other

Required Work

1. Create document metadata tables.
2. Configure Supabase Storage.
3. Create private per-user storage paths.

4. Add Row Level Security.
5. Add file upload.
6. Add file download.
7. Add preview for supported file types.
8. Add rename.
9. Add tags.
10. Add descriptions.
11. Add document categories.
12. Add issue and expiration dates.
13. Add search.
14. Add filters.
15. Link documents to application requirements.
16. Show where each document is used.
17. Add document-replacement flow.
18. Preserve old versions or provide clear replacement behavior.
19. Add file-size limits.
20. Add permitted file-type rules.
21. Add malware-scanning integration placeholder.
22. Add clear warnings for sensitive documents.
23. Avoid displaying sensitive documents in dashboard previews.
24. Add deletion and recovery behavior.

Security Requirements

- Private storage only.
- Signed URLs with short expiration.
- Server-side authorization before access.
- No public buckets.

- No predictable cross-user paths.
- Restrict file types.
- Restrict file size.
- Avoid logging document content.
- Document data-retention rules.

Acceptance Criteria

- Users can securely upload and retrieve their own documents.
- Users cannot access other users' files.
- Documents can be linked to applications.
- Expiring documents are flagged.
- File validation is tested.

Stop after Phase 7.

18. PHASE 8 — RECOMMENDER MANAGEMENT

Objective

Help the user coordinate recommendation letters professionally.

Required Work

1. Create recommender tables.
2. Add recommender profiles.
3. Track relationship context.
4. Track application-specific requests.
5. Track deadlines.
6. Track request status.
7. Track receipt status.
8. Track thank-you status.
9. Link recommenders to applications.

10. Add recommender workload overview.
11. Warn against overusing one recommender.
12. Add request email templates.
13. Add reminder email templates.
14. Add thank-you email templates.
15. Allow users to copy templates.
16. Do not send email yet.
17. Add contact-history notes.
18. Add a recommendation-packet checklist.
19. Allow attaching a CV, personal statement, and scholarship description.
20. Add reminders.

Recommended Statuses

- Potential
- Planning to Ask
- Asked
- Accepted
- In Progress
- Received
- Declined
- No Response
- Thanked

AI Features

AI may draft:

- Initial recommendation request.
- Polite reminder.
- Deadline update.

- Thank-you message.
- Recommender briefing summary.

AI must not misrepresent the relationship or fabricate accomplishments.

Acceptance Criteria

- Recommenders and requests can be tracked.
- A recommender can be linked to multiple applications.
- Upcoming recommendation deadlines are visible.
- Email templates are editable before copying.
- No emails are sent automatically.

Stop after Phase 8.

19. PHASE 9 — EXPERIENCE AND ACHIEVEMENT TRACKER

Objective

Build a master record of experiences that can support scholarship essays and applications.

Experience Categories

- Clinical experience
- Shadowing
- Community service
- Research
- Employment
- Leadership
- Teaching
- Mentoring
- Advocacy
- Student organization

- Publication
- Presentation
- Award
- Certification
- Personal project
- Other

Required Work

1. Create experience tables.
2. Add CRUD interfaces.
3. Add hours.
4. Add dates.
5. Add impact statements.
6. Add leadership indicators.
7. Add clinical indicators.
8. Add research indicators.
9. Add service indicators.
10. Add tags.
11. Add supporting-document links.
12. Add scholarship-use history.
13. Add search.
14. Add filters.
15. Add timeline view.
16. Add total-hour summaries.
17. Add duplicate detection.
18. Add export.
19. Add “story bank” fields.

20. Add reflection fields.

Story Bank Fields

For each meaningful experience, allow the user to record:

- Situation
- Challenge
- Action
- Result
- Lesson
- Impact on others
- Personal growth
- Connection to medicine
- Connection to career goals
- Possible essay themes

AI Features

AI may help:

- Convert notes into a concise activity description.
- Identify measurable impact.
- Suggest scholarship themes.
- Match experiences to essay prompts.
- Identify missing details.

AI may not invent data, hours, outcomes, patient information, or leadership responsibilities.

Acceptance Criteria

- Experiences can be stored and searched.
- Story-bank entries can be linked to essays.
- Hour calculations are accurate.

- AI assistance clearly separates known facts from suggestions.

Stop after Phase 9.

20. PHASE 10 — ELIGIBILITY AND MATCH ENGINE

Objective

Compare user profile information with scholarship requirements.

Required Work

1. Create an explainable rules engine.
2. Avoid relying exclusively on generative AI.
3. Use structured eligibility fields where possible.
4. Calculate:
 - Clearly eligible
 - Likely eligible
 - Possibly eligible
 - Missing information
 - Likely ineligible
5. Show the exact reasons for each result.
6. Allow users to override or correct assumptions.
7. Add eligibility questions.
8. Add missing-profile prompts.
9. Add specialty matching.
10. Add geographic matching.
11. Add school-year matching.
12. Add citizenship matching.
13. Add financial-need matching.
14. Add service-interest matching.

15. Add scholarship-specific custom conditions.

16. Add an eligibility-confidence score.

17. Add tests for rule combinations.

Explainability Example

Display:

“Likely eligible because:

- You are enrolled in a U.S. medical school.
- The scholarship accepts second-year students.
- Ohio residents are eligible.
- No specialty restriction conflicts with your profile.

Needs verification:

- The scholarship requires demonstrated financial need.
- Your profile does not yet include supporting financial information.”

AI Role

AI may assist with extracting eligibility rules from pasted scholarship text.

The extracted rules must:

- Be presented for user review.
- Be marked unverified.
- Include source text.
- Require confirmation before affecting eligibility results.

Acceptance Criteria

- Eligibility decisions are transparent.
- Structured rules are used where possible.
- Users can see missing information.
- AI-extracted rules are never treated as verified automatically.
- Tests cover common edge cases.

Stop after Phase 10.

21. PHASE 11 — APPLICATION PRIORITIZATION AND EXPECTED VALUE

Objective

Help the user allocate limited application time.

Required Calculations

Expected Scholarship Value

A simple expected-value model may use:

Expected value = estimated probability of award × estimated award amount

Because the true probability is usually unknown:

- Label it as an estimate.
- Show assumptions.
- Allow manual adjustment.
- Avoid false precision.
- Use ranges where appropriate.

Return on Time

Possible metric:

Expected value per hour = expected scholarship value ÷ estimated application hours

Readiness Score

Inputs may include:

- Eligibility verified
- Scholarship verified
- Essays available
- Documents available
- Recommenders confirmed
- Tasks completed

- Deadline time remaining

Strategic Value

Allow users to increase strategic value for:

- Renewable scholarships
- Large local scholarships
- Low applicant pools
- Scholarships aligned with career goals
- Scholarships with reusable application materials
- Scholarships that establish future relationships

Required Work

1. Create prioritization settings.
2. Add configurable weights.
3. Add default values.
4. Show factor breakdown.
5. Add comparison view.
6. Add “best next application” view.
7. Add low-effort quick wins.
8. Add high-value opportunities.
9. Add deadline-critical opportunities.
10. Add missing-material warnings.
11. Add scenario analysis.
12. Add sensitivity testing where practical.

Acceptance Criteria

- Scores are explainable.
- Users can change assumptions.
- No score is described as a guarantee.

- Calculations have unit tests.
- Rankings update as application data changes.

Stop after Phase 11.

22. PHASE 12 — FINANCIAL IMPACT AND DEBT PROJECTION

Objective

Show how scholarship awards may affect educational borrowing.

Required Work

1. Create financial assumptions.
2. Allow annual tuition entry.
3. Allow annual fee entry.
4. Allow annual living-expense entry.
5. Allow existing savings entry.
6. Allow family contribution entry.
7. Allow current scholarship entry.
8. Allow expected borrowing entry.
9. Allow interest-rate assumptions.
10. Allow tuition-increase assumptions.
11. Create a baseline without new scholarships.
12. Create a scenario with awarded scholarships.
13. Create a scenario with pending scholarships.
14. Show tuition reduction.
15. Show borrowing avoided.
16. Show projected interest avoided.
17. Show projected graduation debt.
18. Show assumptions clearly.

19. Add year-by-year table.

20. Add charts.

21. Add exportable report.

22. Add disclaimers that calculations are educational estimates.

Scenarios

- No additional scholarships
- Current confirmed scholarships
- Confirmed plus pending applications
- User-defined target
- Conservative
- Expected
- Optimistic

Important Rules

- Do not provide personalized financial-advisor claims.
- Do not hide assumptions.
- Do not mix scholarship awards with loan forgiveness.
- Treat taxable stipends separately where relevant.
- Allow users to exclude scholarships that cannot be stacked.
- Allow the user to model scholarship displacement of institutional aid.

Acceptance Criteria

- Financial calculations reconcile.
- Users can see year-by-year impact.
- Assumptions are editable.
- Charts update.
- Tests cover interest and borrowing calculations.

Stop after Phase 12.

23. PHASE 13 — REPORTS AND ANALYTICS

Objective

Help the user evaluate whether the scholarship strategy is working.

Reports

Application Performance

- Applications started
- Applications submitted
- Awards won
- Win rate
- Average award
- Total awarded
- Total pending
- Time invested

Return on Effort

- Dollars awarded per hour
- Dollars applied for per hour
- Expected value per hour
- Performance by scholarship category
- Performance by award size
- Performance by application effort

Essay Performance

- Essays used most often
- Essays associated with awards
- Average revision count
- Prompts by category

Avoid implying that an essay caused an award.

Recommender Performance

Track operational data such as:

- Response time
- Completion rate
- Number of requests

Do not rank recommenders as people or claim one caused an award.

Deadline Performance

- On-time submission rate
- Late or missed opportunities
- Average days started before deadline
- Applications completed under deadline pressure

Funding Progress

- Annual goal
- Total awarded
- Remaining target
- Projected tuition reduction
- Projected debt reduction

Acceptance Criteria

- Reports are filterable by year.
- Charts have accessible labels.
- Empty states are clear.
- Data can be exported.
- Metrics are documented.

Stop after Phase 13.

24. PHASE 14 — NOTIFICATIONS AND CALENDAR INTEGRATION

Objective

Reduce missed deadlines.

Required Work

1. Build notification preferences.
2. Add in-app notifications.
3. Add email-notification infrastructure.
4. Add daily digest.
5. Add weekly review.
6. Add deadline reminders.
7. Add recommender reminders.
8. Add stale-scholarship reminders.
9. Add renewal reminders.
10. Add expiring-document reminders.
11. Add Google Calendar integration.
12. Add Microsoft Outlook Calendar integration if practical.
13. Support adding:
 - Scholarship deadlines
 - Task deadlines
 - Recommendation deadlines
 - Renewal deadlines
14. Handle timezone correctly.
15. Prevent duplicate calendar events.
16. Allow users to disconnect integrations.
17. Document OAuth setup.

Reminder Defaults

Suggested reminders:

- 60 days before
- 30 days before
- 14 days before
- 7 days before
- 3 days before
- 1 day before

Users must be able to customize these.

Acceptance Criteria

- Notifications respect user settings.
- Calendar events are not duplicated.
- Timezones are handled correctly.
- Disconnecting integrations works.
- Failures are logged safely and shown clearly.

Stop after Phase 14.

25. PHASE 15 — AI SCHOLARSHIP INTAKE

Objective

Allow the user to paste scholarship information and convert it into a structured draft record.

Input Methods

- Paste webpage text.
- Paste scholarship announcement.
- Paste email.
- Paste PDF text.
- Upload document for extraction later.

- Enter URL for metadata assistance where legally and technically appropriate.

Required Work

1. Build scholarship text intake form.
2. Send text to the AI extraction service.
3. Extract:
 - Name
 - Organization
 - Award amount
 - Deadline
 - Eligibility
 - Required documents
 - Essay prompts
 - Recommendation requirements
 - Renewal information
 - Service obligations
 - Official links
4. Display side-by-side:
 - Source text
 - Extracted fields
5. Require user confirmation.
6. Mark all extracted data unverified.
7. Save extraction confidence per field.
8. Highlight uncertain fields.
9. Preserve source text.
10. Create follow-up verification tasks.
11. Prevent the AI from guessing missing details.

12. Add extraction tests using sample scholarship announcements.

Acceptance Criteria

- Pasted text produces a structured draft.
- The user reviews every field before saving.
- Missing information stays blank.
- Uncertain data is highlighted.
- Source text remains available.
- AI output never becomes officially verified automatically.

Stop after Phase 15.

26. PHASE 16 — SCHOLARSHIP DISCOVERY ASSISTANT

Objective

Help identify potential opportunities without creating an unreliable uncontrolled scraper.

Begin with a human-reviewed discovery workflow.

Required Work

1. Create discovery-source records.
2. Allow users to save source organizations.
3. Add source categories:
 - Medical associations
 - Specialty societies
 - State medical societies
 - County medical societies
 - Community foundations
 - Hospital foundations
 - Universities
 - Employers

- Civic organizations
 - Religious organizations
 - Disease foundations
 - Professional associations
4. Add source-review schedules.
 5. Add saved search queries.
 6. Add a discovery inbox.
 7. Allow opportunity links to be saved quickly.
 8. Allow pasted results to use the AI intake process.
 9. Add duplicate detection.
 10. Add official-source preference.
 11. Add search-provider abstraction.
 12. Add rate limiting.
 13. Respect terms of service and robots rules.
 14. Do not scrape websites that prohibit automated access.
 15. Do not bypass authentication, CAPTCHAs, or paywalls.
 16. Do not automatically claim that a scholarship is active.
 17. Create manual verification tasks.
 18. Add source-quality scoring.
 19. Add “last checked” date.
 20. Add stale-source warnings.

Source Quality Levels

- Official scholarship page
- Sponsoring organization page
- Medical-school financial-aid page
- Reputable scholarship directory

- Secondary reference
- Unverified lead

Acceptance Criteria

- Users can build a discovery-source library.
- New leads enter a review inbox.
- Leads are deduplicated.
- No unverified lead is treated as active automatically.
- Source and verification status remain visible.

Stop after Phase 16.

27. PHASE 17 — EMAIL INTEGRATION

Objective

Support communication while preserving user control.

Required Work

1. Add Gmail integration.
2. Add Microsoft Outlook integration if practical.
3. Allow users to draft:
 - Recommender requests
 - Reminder messages
 - Thank-you messages
 - Scholarship-organization questions
4. Show recipients, subject, and body before sending.
5. Require explicit confirmation.
6. Store communication metadata.
7. Avoid storing full email bodies unless necessary.
8. Track sent date.

9. Track response manually or through authorized metadata.
10. Allow integration disconnection.
11. Document OAuth scopes.
12. Request the minimum permissions necessary.

Prohibited Behavior

- No autonomous email sending.
- No bulk outreach without review.
- No impersonation.
- No fabricated relationship claims.
- No collection of unrelated mailbox content.

Acceptance Criteria

- A user can generate and review a draft.
- Sending requires explicit action.
- OAuth scopes are minimal.
- Email failures are handled clearly.

Stop after Phase 17.

28. PHASE 18 — ADMINISTRATION AND MULTI-USER READINESS

Objective

Prepare the system for broader use without compromising privacy.

Required Work

1. Define user roles.
2. Add admin role.
3. Add scholarship curator role.
4. Add support role if necessary.
5. Add shared scholarship moderation.

6. Add audit logs.
7. Add abuse protection.
8. Add rate limiting.
9. Add account suspension tools.
10. Add scholarship-reporting workflow.
11. Add incorrect-information reporting.
12. Add database backup strategy.
13. Add disaster-recovery documentation.
14. Add data export.
15. Add account deletion.
16. Add privacy-policy placeholder.
17. Add terms-of-use placeholder.
18. Add consent for AI processing.
19. Add data-retention settings.
20. Add security review checklist.

Acceptance Criteria

- Roles are enforced server-side.
- Admins cannot casually access private essays or documents.
- Sensitive access is audited.
- Users can export and delete their data.
- Shared scholarship updates are moderated.

Stop after Phase 18.

29. PHASE 19 — PRODUCTION HARDENING

Objective

Prepare the application for dependable real-world use.

Required Work

1. Conduct a full security review.
2. Review Row Level Security.
3. Review storage policies.
4. Review authorization checks.
5. Review secrets management.
6. Add rate limiting.
7. Add structured logging.
8. Add error monitoring.
9. Add performance monitoring.
10. Add database indexes.
11. Review query performance.
12. Add loading skeletons.
13. Add graceful degradation.
14. Add accessibility audit.
15. Add mobile responsiveness audit.
16. Add cross-browser testing.
17. Add backup procedures.
18. Add restore procedures.
19. Add release checklist.
20. Add production deployment guide.
21. Add staging environment.
22. Add CI/CD.
23. Add migration safety procedures.
24. Add seed-data procedures.
25. Add user documentation.

26. Add administrator documentation.
27. Add known-limitations documentation.
28. Add incident-response outline.

Test Coverage

Include:

- Unit tests
- Component tests
- Integration tests
- End-to-end tests
- Authorization tests
- Row Level Security tests
- Financial-calculation tests
- Ranking tests
- AI-output validation tests
- File-access tests
- Calendar integration tests
- Email confirmation tests

Acceptance Criteria

- All critical flows pass.
- No known critical security issue remains.
- Production deployment succeeds.
- Monitoring works.
- Backups are documented.
- User data isolation is verified.
- Accessibility issues are addressed.

Stop after Phase 19.

30. INITIAL DASHBOARD DESIGN

The dashboard should be visually clean and not overwhelming.

Top Summary Cards

- Awarded
- Pending
- Applied For
- Tuition Reduction
- Deadlines This Month

Main Dashboard Layout

Left Main Column

- Recommended next actions
- Priority applications
- Upcoming deadlines
- Applications at risk

Right Column

- Funding goal progress
- Application pipeline summary
- Profile readiness
- Essay readiness
- Document readiness

Lower Section

- Recent activity
- Monthly application progress
- Scholarship categories
- Financial projection preview

Use progressive disclosure so advanced data does not overwhelm a new user.

31. DESIGN SYSTEM

The interface should feel:

- Professional
- Calm
- Modern
- Trustworthy
- Organized
- Motivating
- Appropriate for medical students

Avoid:

- Childish gamification
- Excessive gradients
- Excessive animations
- Dense screens
- Overly colorful charts
- Unnecessary medical imagery
- Misleading success claims

Design Requirements

- Accessible contrast
- Clear focus states
- Responsive layout
- Mobile-friendly tables
- Keyboard accessibility
- Screen-reader labels

- Consistent form design
 - Clear destructive-action confirmations
 - Helpful empty states
 - Search-first navigation where appropriate
 - Command palette later if useful
-

32. SCHOLARSHIP CATEGORIES

Create a flexible category system that supports:

- Institutional
- National
- State
- County
- Local community
- Medical society
- Specialty society
- Hospital foundation
- Community foundation
- Employer-sponsored
- Professional association
- Civic organization
- Religious organization
- Research
- Leadership
- Service
- Rural health
- Primary care

- Financial need
 - Merit
 - Diversity and perspective
 - Military family
 - Disease foundation
 - Alumni
 - Memorial scholarship
 - Other
-

33. SAMPLE USER WORKFLOW

Design the system so the following workflow is easy:

1. User creates an account.
2. User completes a scholarship profile.
3. User adds tuition and debt assumptions.
4. User imports a list of scholarships.
5. User reviews eligibility matches.
6. User selects promising scholarships.
7. The system creates application records.
8. The system creates default tasks.
9. The user uploads a transcript and CV.
10. The user adds recommenders.
11. The user links recommenders to applications.
12. The user builds reusable essays.
13. The user adapts essays to specific prompts.
14. The dashboard prioritizes upcoming applications.
15. The user submits an application.

16. The user records the submission confirmation.

17. The user records the result.

18. Award totals update.

19. Debt projections update.

20. Reports show return on effort.

34. SEED DATA

Create development seed data with fictional scholarships only.

Do not use real scholarship names unless their current details have been verified separately.

Sample fictional records should include:

- A local county medical society scholarship
- A statewide primary-care scholarship
- A national leadership scholarship
- A rural health scholarship
- A hospital-foundation scholarship
- A renewable community-service scholarship
- A research scholarship
- A scholarship requiring two recommendation letters
- A scholarship requiring financial-need documentation
- A scholarship with a service obligation
- An expired scholarship
- An unverified scholarship

Use clearly fictional organization names.

35. SECURITY AND PRIVACY REQUIREMENTS

Treat the following as sensitive:

- Financial information
- Tax documents
- FAFSA-related information
- Essays about hardship
- Identity documents
- Transcripts
- Recommendation letters
- Personal demographic information
- Contact information
- Career and educational records

Required Controls

- Encryption in transit
- Secure storage
- Row Level Security
- Least-privilege permissions
- Private file buckets
- Short-lived signed URLs
- Server-side authorization
- CSRF protection where relevant
- Input validation
- Output encoding
- Rate limiting
- Secret management
- Audit logging
- Account deletion

- Data export
- Clear retention policies

Do not describe the platform as HIPAA compliant. This is not a healthcare-record system.

36. AI ARCHITECTURE REQUIREMENTS

Create an AI service layer with:

- Provider interface
- Prompt templates
- Structured-output validation
- Retry logic
- Timeouts
- Rate limiting
- Cost tracking
- User-visible failure messages
- Audit metadata
- Model configuration
- Feature flags
- Token limits
- Redaction options

Structured Outputs

Use schemas for:

- Scholarship extraction
- Eligibility-rule extraction
- Essay feedback
- Essay adaptation
- Experience summarization

- Task recommendations
- Recommender email drafting

Validate every AI response before displaying it as structured data.

AI Disclosure

Display notices such as:

- “AI-generated suggestion”
 - “Verify against the official scholarship source”
 - “Do not submit without reviewing”
 - “The AI may misunderstand eligibility rules”
 - “The AI must not invent personal experiences”
-

37. FINANCIAL CALCULATION REQUIREMENTS

Create a dedicated calculation module.

It should support:

- Tuition by year
- Tuition inflation
- Fees
- Living costs
- Existing aid
- Scholarship awards
- Award restrictions
- Renewable awards
- Loan disbursement timing
- Interest accrual
- Capitalization
- Repayment term

- Scholarship displacement
- User-defined scenarios

Every result must include:

- Formula or methodology description
- Assumptions
- Date calculated
- Disclaimer

Avoid excessive decimal precision.

38. TESTING REQUIREMENTS BY PHASE

Every phase must include tests relevant to its features.

At minimum:

Unit Tests

- Validation
- Scoring
- Financial calculations
- Eligibility rules
- Deadline-risk logic
- Word-count logic
- Application-state transitions

Integration Tests

- Authentication
- Database operations
- Row Level Security
- File access
- Application creation

- Scholarship imports
- AI structured-output validation

End-to-End Tests

- User onboarding
- Scholarship creation
- Application creation
- Task completion
- Essay versioning
- Document upload
- Recommendation tracking
- Award recording
- Dashboard update

Do not treat test-writing as optional.

39. DOCUMENTATION REQUIREMENTS

Maintain:

- README
- Product requirements
- Architecture overview
- Database schema
- Environment-variable guide
- Supabase setup
- Local-development guide
- Testing guide
- Deployment guide
- Security guide

- AI configuration guide
- User guide
- Admin guide
- Troubleshooting guide
- Changelog
- Roadmap
- Known limitations

Update documentation at the end of each phase.

40. GIT AND VERSION CONTROL

Use a disciplined Git workflow.

For each phase:

1. Create a branch.
2. Make logical commits.
3. Use descriptive commit messages.
4. Avoid committing secrets.
5. Update the changelog.
6. Tag stable milestones where appropriate.

Suggested branch naming:

- phase-0-foundation
- phase-1-auth-profile
- phase-2-scholarships
- phase-3-applications

Suggested commit style:

- feat: add scholarship creation form
- fix: enforce user ownership in application query

- test: add scholarship filter coverage
 - docs: update Supabase setup instructions
-

41. ENVIRONMENT VARIABLES

Create a `.env.example` that eventually supports:

- Supabase URL
- Supabase anonymous key
- Supabase service-role key
- Application URL
- Anthropic API key
- OpenAI API key
- Google OAuth credentials
- Microsoft OAuth credentials
- Email provider credentials
- Error-monitoring credentials

Never expose service-role keys or AI keys to the browser.

42. DEPLOYMENT STRATEGY

Use:

- GitHub for source control
- Vercel for application hosting
- Supabase for database, authentication, and storage
- GitHub Actions for testing
- Staging and production environments

The deployment guide should explain:

1. Creating the Supabase project.

2. Applying migrations.
 3. Creating storage buckets.
 4. Configuring authentication.
 5. Configuring environment variables.
 6. Deploying to Vercel.
 7. Running seed data.
 8. Configuring a custom domain.
 9. Configuring monitoring.
 10. Rolling back a failed release.
-

43. DO NOT BUILD YET

Do not build the following unless requested in a later project:

- Medical billing
- Electronic health records
- Patient records
- Clinical decision support
- DPC membership billing
- Residency interview management
- ERAS submission automation
- Medical licensing management
- CME tracking
- Practice payroll
- Patient scheduling
- Medical advice tools
- Automated scholarship submission
- Uncontrolled web scraping

- Autonomous email sending
-

44. PHASE COMPLETION TEMPLATE

At the end of each phase, respond in this structure:

Phase Completed

State the phase name.

Features Implemented

List the completed functionality.

Files Added or Modified

Summarize important files.

Database Changes

List migrations, tables, indexes, policies, and seed changes.

Security Work

List authorization, validation, and privacy controls added.

Tests

List tests and results.

Commands Run

Include:

- Install
- Lint
- Type check
- Unit tests
- End-to-end tests
- Build

Manual Setup Required

List any user actions.

Known Limitations

Be transparent.

Next Phase

Describe the next phase but do not begin it without approval.

45. INITIAL INSTRUCTION

Begin with Phase 0 only.

Before writing code:

1. Inspect the current project directory.
2. State whether the directory is empty or already contains a project.
3. Identify any conflicts with this specification.
4. Create a concise implementation plan for Phase 0.
5. Then implement Phase 0.
6. Run all relevant checks.
7. Fix errors.
8. Provide the Phase Completion report.
9. Stop.

Do not begin Phase 1 until I explicitly tell you:

“Proceed to Phase 1.”

46. LATER-PHASE COMMANDS

When I say:

“Proceed to the next phase,”

you should:

1. Review the current codebase.
2. Review the master specification.

3. Confirm the next unfinished phase.
4. Inspect existing architecture and migrations.
5. Create a concise implementation plan.
6. Implement only that phase.
7. Test it thoroughly.
8. Update documentation.
9. Provide the completion report.
10. Stop for approval.

When I report a bug:

1. Reproduce it if possible.
2. Identify the root cause.
3. Explain the affected area.
4. Fix it with the smallest reliable change.
5. Add a regression test.
6. Run relevant tests.
7. Report the result.

When I request a design change:

1. Preserve existing functionality.
2. Avoid unnecessary rewrites.
3. Update affected tests.
4. Confirm accessibility.
5. Report modified files.

47. FINAL PRODUCT SUCCESS CRITERIA

The completed application should allow a medical student to:

- Maintain a verified scholarship database.

- Track every scholarship application.
- See deadlines and required materials.
- Prioritize opportunities.
- Reuse and adapt essays.
- Manage documents securely.
- Coordinate recommenders.
- Track experiences and achievements.
- Understand eligibility.
- Estimate expected value.
- Record scholarship awards.
- Measure debt reduction.
- Review performance over time.
- Receive reminders.
- Use AI without losing control or authenticity.
- Export personal data.
- Trust that private data is isolated.

The system should be reliable enough to use throughout all four years of medical school.

Begin Phase 0 now.